API REFERENCE

City Climate Data REST API

Field	Value
Version	1.0.0
Base URL	http://127.0.0.1:8000/api/
Protocol	HTTP/1.1
Data Format	JSON (application/json)
Authentication	None (development build)
Module	XJCO3011 Web Services and Web Data
Last Updated	April 2026

This document describes every endpoint, request parameter, request body field, and **response schema** for the City Climate Data REST API. Example request/response pairs are included for every operation. All responses are **JSON** unless otherwise stated.

1. Overview

1.1 Base URL

All endpoints described in this document are relative to the base URL:

```
http://127.0.0.1:8000/api/
```

1.2 Resources

The API exposes two primary resources and a set of analytics / live-data endpoints:

Resource	Path Prefix	Description
City	/api/cities/	Urban location metadata (name, country, coordinates)
WeatherRecord	/api/weather/	Per-city, per-date climate measurements
Analytics — Temperature	/api/analytics/temperature-trend/	Monthly temperature aggregates for a city
Analytics — Air Quality	/api/analytics/air-quality-anomaly/	Days where PM2.5 exceeds a configurable threshold
Live Weather	/api/weather/live/	Fetch today's live data from OpenWeatherMap and upsert

1.3 Content Type

All requests that include a body must set the Content-Type header:

```
Content-Type: application/json
```

All successful responses return Content-Type: application/json.

1.4 Authentication

This version of the API does not require authentication. All endpoints are publicly accessible. A future version will implement Token-based authentication via the Authorization: Token <token> header.

1.5 HTTP Status Codes

The API uses standard HTTP status codes consistently:

Code	Meaning	When it occurs
200 OK	Success	GET, PUT, PATCH returned a resource or list
201 Created	Resource created	POST successfully created a new resource
204 No Content	Deleted	DELETE succeeded; no body returned

Code	Meaning	When it occurs
400 Bad Request	Invalid input	Missing required query parameter or malformed body
404 Not Found	Resource missing	Requested ID does not exist, or no records for given city
503 Service Unavailable	Upstream error	OpenWeatherMap API call failed (network / invalid key)

2. Data Models

2.1 City

Represents a geographic location. Used as a foreign key in WeatherRecord.

Field	Type	Constraints	Description
id	integer	Auto, PK	Unique identifier (auto-incremented)
name	string	max 100 chars, unique	City name, e.g. "London"
country	string	max 50 chars	ISO country code or full name, e.g. "GB"
latitude	float	required	Geographic latitude in decimal degrees
longitude	float	required	Geographic longitude in decimal degrees

Example City object:

```
{
  "id": 1,
  "name": "London",
  "country": "GB",
  "latitude": 51.5074,
  "longitude": -0.1278
}
```

2.2 WeatherRecord

Stores a single day's climate measurements for one city. The combination of (city, date) is unique — at most one record per city per day.

Field	Type	Constraints	Description
id	integer	Auto, PK	Unique identifier (auto-incremented)
city	integer (FK)	required	ID of the related City
date	string (date)	required, unique with city	ISO 8601 date, e.g. "2024-06-15"
temperature	float	required	Air temperature in degrees Celsius (°C)
humidity	integer	required	Relative humidity as a percentage (0–100)
pm25	float	optional (nullable)	PM2.5 concentration in µg/m³; null if unavailable
wind_speed	float	required	Wind speed in metres per second (m/s)

Example WeatherRecord object:

```
{
  "id": 42,
  "city": 1,
  "date": "2024-06-15",
  "temperature": 22.5,
  "humidity": 60,
```

```
  "pm25": 45.0,
  "wind_speed": 3.2
}
```

3. City Endpoints

GET `/api/cities/` — *List all cities*

Returns an array of all City objects in the database. No query parameters required.

Request

Example request

```
GET /api/cities/ HTTP/1.1
Host: 127.0.0.1:8000
```

Response — 200 OK

```
[
  {
    "id": 1,
    "name": "London",
    "country": "GB",
    "latitude": 51.5074,
    "longitude": -0.1278
  },
  {
    "id": 2,
    "name": "Beijing",
    "country": "CN",
    "latitude": 39.9042,
    "longitude": 116.4074
  }
]
```

200 Array of City objects (may be empty [])

POST `/api/cities/` — *Create a new city*

Request Body Fields

Field	Type	Required	Description
name	string	Yes	City name (must be unique)
country	string	Yes	Country code or name
latitude	float	Yes	Decimal latitude
longitude	float	Yes	Decimal longitude

Example request

```
POST /api/cities/ HTTP/1.1
Content-Type: application/json

{
  "name": "Shanghai",
  "country": "CN",
  "latitude": 31.2304,
  "longitude": 121.4737
}
```

Response — 201 Created

```
{
  "id": 3,
```

```
  "name": "Shanghai",
  "country": "CN",
  "latitude": 31.2304,
  "longitude": 121.4737
}
```

201 Newly created City object
400 Validation error — e.g. name already exists or missing field

GET `/api/cities/{id}/` — *Retrieve a single city*

PUT `/api/cities/{id}/` — *Full update of a city*

PATCH `/api/cities/{id}/` — *Partial update of a city*

DELETE `/api/cities/{id}/` — *Delete a city and all its weather records*

Path Parameter

Parameter	Type	Description
id	integer	The city's primary key

Example requests

```
GET /api/cities/1/ HTTP/1.1

PATCH /api/cities/1/ HTTP/1.1
Content-Type: application/json
{"name": "Greater London"}

DELETE /api/cities/1/ HTTP/1.1
```

Responses

200 City object (GET / PUT / PATCH)
204 No body (DELETE succeeded)
400 Validation error (PUT / PATCH)
404 City with given id not found

4. Weather Record Endpoints

GET `/api/weather/` — *List all weather records*

Returns all WeatherRecord objects, ordered by date descending (most recent first).

Example request

```
GET /api/weather/ HTTP/1.1
```

Response — 200 OK

```
[
  {
    "id": 42,
    "city": 1,
    "date": "2024-06-15",
    "temperature": 22.5,
    "humidity": 60,
    "pm25": 45.0,
    "wind_speed": 3.2
  },
  ...
]
```

POST `/api/weather/` — *Create a new weather record*

Request Body Fields

Field	Type	Required	Description
city	integer	Yes	ID of the related City
date	string	Yes	ISO 8601 date, e.g. "2024-06-15"
temperature	float	Yes	Temperature in °C
humidity	integer	Yes	Relative humidity (0–100)
pm25	float	No	PM2.5 in µg/m³; omit or send null if unknown
wind_speed	float	Yes	Wind speed in m/s

Example request

```
POST /api/weather/ HTTP/1.1
Content-Type: application/json

{
  "city": 1,
  "date": "2024-06-16",
  "temperature": 25.0,
  "humidity": 55,
  "pm25": 30.0,
  "wind_speed": 2.0
}
```

Responses

- **201** Newly created WeatherRecord object
- **400** Validation error (missing field, wrong type, or duplicate city+date)

GET `/api/weather/{id}/` — *Retrieve a weather record*

PUT `/api/weather/{id}/` — *Full update*

PATCH `/api/weather/{id}/` — *Partial update*

DELETE `/api/weather/{id}/` — *Delete a weather record*

Path Parameter

Parameter	Type	Description
id	integer	The weather record's primary key

Example partial update

```
PATCH /api/weather/42/ HTTP/1.1
Content-Type: application/json
{"temperature": 20.0}
```

Responses

- **200** WeatherRecord object (GET / PUT / PATCH)
- **204** No body (DELETE succeeded)
- **400** Validation error
- **404** Record with given id not found

5. Live Weather Endpoint

GET `/api/weather/live/` — *Fetch & upsert today's live weather from OpenWeatherMap*

Calls the OpenWeatherMap Current Weather and Air Pollution APIs using the city's stored latitude/longitude. Creates or updates the WeatherRecord for today's date (upsert). Requires OPENWEATHER_API_KEY to be set in the server environment.

Query Parameters

Parameter	Type	Required	Default	Description
city_id	integer	Yes	—	ID of the City to fetch live weather for

Example request

```
GET /api/weather/live/?city_id=1 HTTP/1.1
```

Response — 200 OK

Returns the upserted WeatherRecord plus an additional source field:

```
{
  "id": 99,
  "city": 1,
  "date": "2026-04-17",
  "temperature": 14.3,
  "humidity": 72,
  "pm25": 18.6,
  "wind_speed": 5.1,
  "source": "openweathermap"
}
```

Error Responses

- **400** city_id query parameter is missing
- **404** No City found with the given city_id
- **503** OpenWeatherMap API call failed (network error, invalid key, or rate limit)

Example 400 response:

```
{
  "error": "请提供 city_id 参数"
}
```

Example 503 response:

```
{
  "error": "Invalid or missing OpenWeather API key (HTTP 401)."
}
```

6. Analytics Endpoints

6.1 Monthly Temperature Trend

GET `/api/analytics/temperature-trend/` — *Monthly temperature & PM2.5 aggregates for a city*

Queries all WeatherRecord rows for the specified city, groups them by (year, month), and returns monthly aggregates: average, maximum, and minimum temperature, plus average PM2.5.

Query Parameters

Parameter	Type	Required	Default	Description
city_id	integer	Yes	—	ID of the City to analyse

Example request

```
GET /api/analytics/temperature-trend/?city_id=1 HTTP/1.1
```

Response — 200 OK

Array of monthly aggregate objects, ordered chronologically:

```
[
  {
    "year": 2024,
    "month": 3,
    "avg_temperature": 11.0,
    "max_temperature": 14.5,
    "min_temperature": 7.2,
    "avg_pm25": 41.0
  },
  {
    "year": 2024,
    "month": 4,
    "avg_temperature": 15.3,
    "max_temperature": 20.1,
    "min_temperature": 11.0,
    "avg_pm25": 38.5
  }
]
```

Response Fields

Field	Type	Description
year	integer	Calendar year
month	integer	Calendar month (1 = January, 12 = December)
avg_temperature	float	Average temperature for the month (°C, 1 d.p.)
max_temperature	float	Maximum temperature recorded that month (°C, 1 d.p.)
min_temperature	float	Minimum temperature recorded that month (°C, 1 d.p.)
avg_pm25	float	Average PM2.5 for the month (µg/m³, 1 d.p.)

Error Responses

400 city_id query parameter is missing

404 City found but has no WeatherRecord rows

6.2 Air Quality Anomaly Detection

GET `/api/analytics/air-quality-anomaly/` — *Days where PM2.5 exceeds a threshold*

Filters WeatherRecord rows for the specified city to those where PM2.5 is strictly greater than the threshold. Returns the count and the full record list, ordered by date descending.

Query Parameters

Parameter	Type	Required	Default	Description
city_id	integer	Yes	—	ID of the City to analyse
threshold	integer	No	75	PM2.5 threshold in µg/m³ (WHO interim target II); days above this are flagged as anomalous

Example request

```
GET /api/analytics/air-quality-anomaly/?city_id=1&threshold=75 HTTP/1.1
```

Response — 200 OK

```
{
  "city_id": "1",
  "threshold": 75,
  "anomaly_count": 2,
  "anomalies": [
    {
      "id": 87,
      "city": 1,
      "date": "2024-01-15",
      "temperature": 4.2,
      "humidity": 80,
      "pm25": 120.5,
      "wind_speed": 1.2
    },
    {
      "id": 63,
      "city": 1,
      "date": "2023-12-22",
      "temperature": 2.1,
      "humidity": 88,
      "pm25": 98.0,
      "wind_speed": 0.8
    }
  ]
}
```

Response Fields

Field	Type	Description
city_id	string	The city_id query parameter echoed back
threshold	integer	The PM2.5 threshold used for filtering
anomaly_count	integer	Number of days where PM2.5 > threshold

Field	Type	Description
anomalies	array	Array of full WeatherRecord objects for anomalous days, newest first

Error Responses

400 city_id query parameter is missing

7. Error Response Format

All error responses return a JSON object with a single error key:

```
{
  "error": "<human-readable error description>"
}
```

DRF validation errors (400) may additionally return field-level details:

```
{
  "name": ["city with this name already exists."],
  "latitude": ["This field is required."]
}
```

7.1 Complete Error Code Reference

Status	Endpoint(s)	Trigger Condition
400	All	Missing required query parameter (city_id) or missing/invalid body field
400	POST /api/cities/	Duplicate city name (unique constraint violation)
400	POST /api/weather/	Duplicate (city, date) combination
404	GET|PUT|PATCH|DELETE /api/cities/{id}/	City with given id does not exist
404	GET|PUT|PATCH|DELETE /api/weather/{id}/	WeatherRecord with given id does not exist
404	GET /api/weather/live/	city_id does not match any City row
404	GET /api/analytics/temperature-trend/	City exists but has zero WeatherRecord rows
503	GET /api/weather/live/	OpenWeatherMap returned non-200; network timeout; missing API key

8. Quick-Start Example

The following sequence demonstrates a complete workflow using curl:

```
# 1. Create a city
curl -X POST http://127.0.0.1:8000/api/cities/ \
     -H "Content-Type: application/json" \
     -d '{"name":"London","country":"GB","latitude":51.5074,"longitude":-0.1278}'

# 2. Add a weather record
curl -X POST http://127.0.0.1:8000/api/weather/ \
     -H "Content-Type: application/json" \
     -d '{"city":1,"date":"2024-06-
15","temperature":22.5,"humidity":60,"pm25":45.0,"wind_speed":3.2}'

# 3. Fetch temperature trend
curl http://127.0.0.1:8000/api/analytics/temperature-trend/?city_id=1
```

```
# 4. Check air quality anomalies (PM2.5 > 75)
curl "http://127.0.0.1:8000/api/analytics/air-quality-
anomaly/?city_id=1&threshold=75"

# 5. Fetch today's live weather (requires OPENWEATHER_API_KEY in .env)
curl "http://127.0.0.1:8000/api/weather/live/?city_id=1"
```

Note: To run the development server, navigate to the `climate_api/` directory and execute `python manage.py runserver`. See the project README for full setup instructions.